

Защищено:
Большаков С.А.

Демонстрация ЛР:
Большаков С.А.

"__" _____ 2025 г.

"__" _____ 2025 г.

**Отчет по лабораторной работе № 5 по курсу
Системное программирование**

" Ввод/вывод в адреса и числа"

(есть ли дополнительные требования - ДА)

26

(количество листов)

Вариант № <10>

ИСПОЛНИТЕЛЬ:

студент группы **ИУ5-42Б**

Кудабаева Э.

(подпись)

"13" апреля 2025 г.

СОДЕРЖАНИЕ

1. Цель выполнения лабораторной работы № 5.....	3
2. Порядок и условия проведения работы № 5.....	3
3. Описание ошибок, возникших при отладке № 5.....	3
4. Блок-схема программы.....	4
5. Скриншот программы КН № 5 в TD.exe	6
6. Текст программы на языке Ассемблера.....	6
7. Результаты работы программы	14
8. Выводы по ЛР № 5	26

1. Цель выполнения лабораторной работы № 5

Целью лабораторной работы №5 является освоение работы с массивами и строками в языке Ассемблер, включая использование буферизации ввода, команд цепочек (LODSB, STOSB, MOVSB), а также преобразование символов в их шестнадцатеричное представление.

2. Порядок и условия проведения работы № 5

Разработать и отладить программу на языке Ассемблер для ввода и буферизации строки символов с клавиатуры (последовательности символов) и затем последовательного их вывода на экран в шестнадцатеричном представлении (через пробел). В данной программе для корректной работы необходимо предусмотреть запоминание строки символов в байтовом массиве. Программа и блок-схема должны содержать вложенные циклы (двойные циклы).

Признак завершения ввода отдельной строки с клавиатуры – это символ "\$" (он вводится с клавиатуры для завершения ввода строки). Между введенной строкой символов и их шестнадцатеричным представлением должен располагаться знак равенства ("="). Максимальное число вводимых символов не должно превышать 20-ти. В данной программе цикл ввода (с клавиатуры) организуется с помощью команд условного (JE, JNE) перехода и команды безусловного перехода (JMP). После завершения ввода строки выполняется ее автоматический вывод. Организовать цикл ввода строк до ввода специального символа (*'). Пример результата работы одного цикла программы показан ниже:

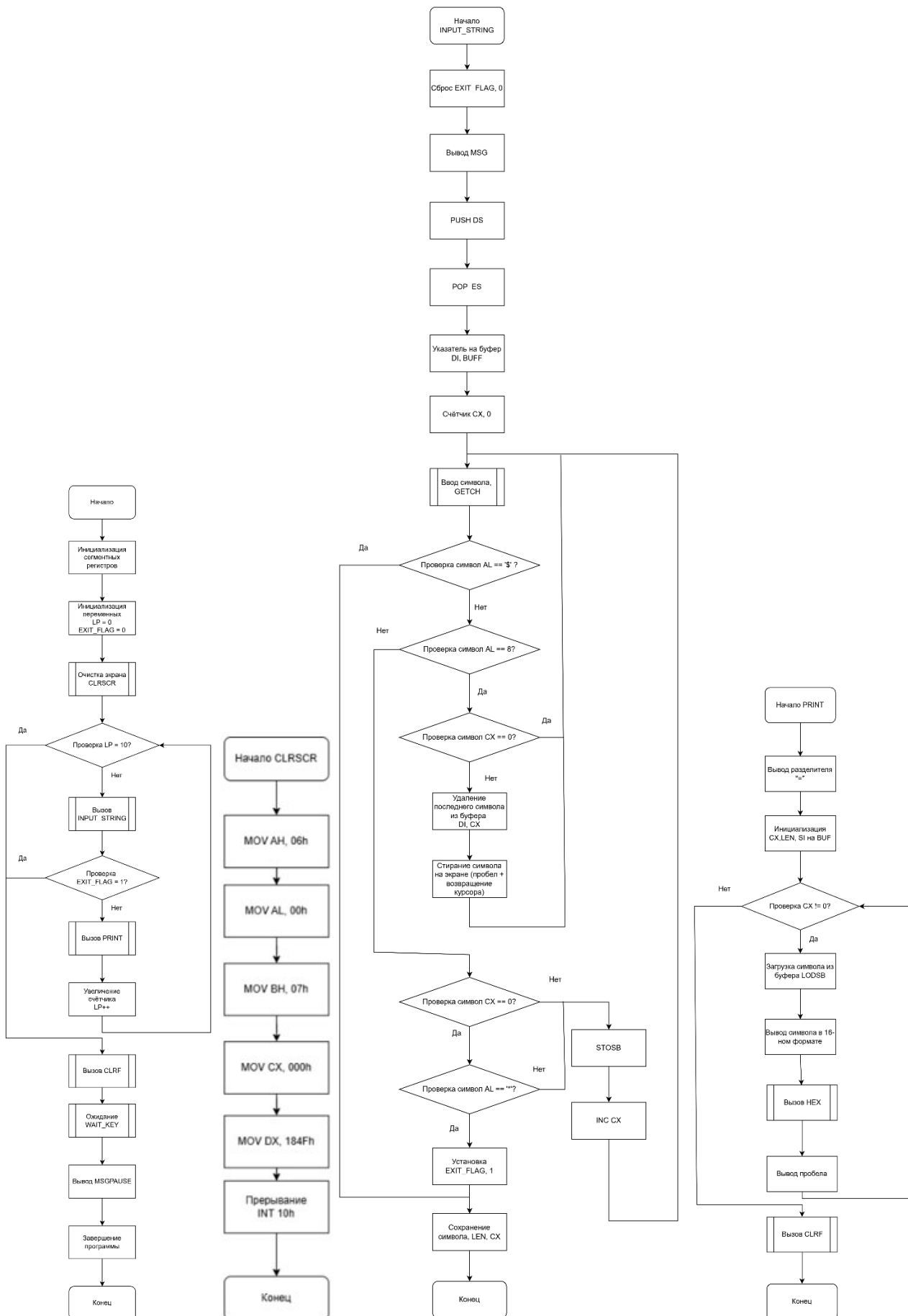
АБВ\$ = 80 81 82

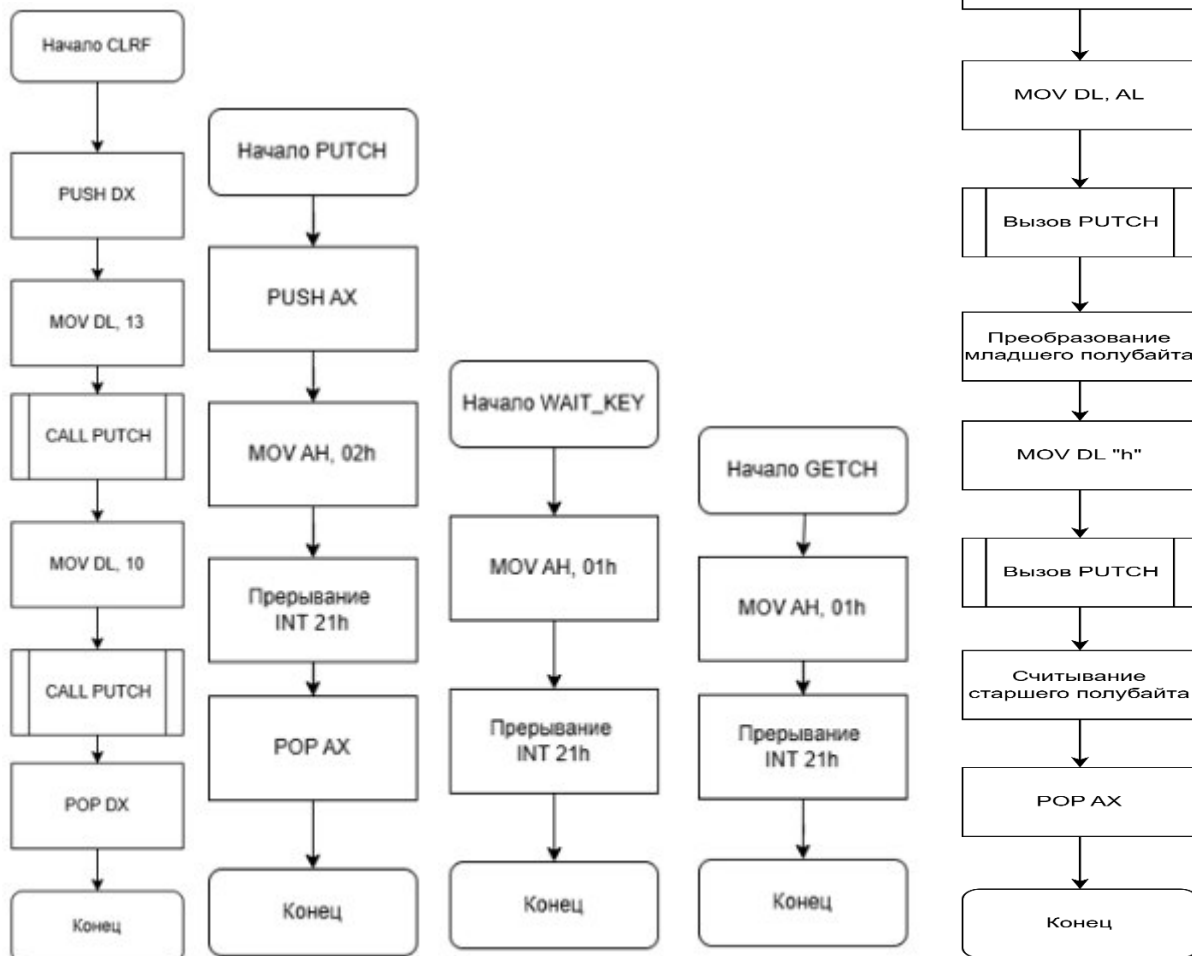
Оформить отчет по ЛР.

3. Описание ошибок, возникших при отладке № 5

Ошибки отсутствуют.

4. Блок-схема программы







DW 100h DUP(?) ; Определение сегмента стека размером 256 слов (512 байт),
инициализированных неопределёнными значениями

DTSEG SEGMENT 'DATA'

; Лабораторная работа №5, Кудабаява Э., Группа ИУ5-42Б

; Сообщения для пользователя

MSG DB 'Введите любую строку максимальной длины 20 символов: \$' ; Приглашение для ввода строки

MSGPAUSE DB 'Нажмите любую клавишу, чтобы завершить программу: '\$' ; Сообщение перед завершением программы

; Буфер для хранения вводимой строки (20 символов, заполнен символами ' ' по умолчанию)

BUF DB 20 DUP('_')

; Переменные для хранения данных

LEN DW 0 ; Длина введённой строки (в словах)

LP DB 0 ; Счётчик строк (количество обработанных строк, максимум 10)

; Таблица для перевода чисел в шестнадцатеричный формат

TABLHEX DB '0123456789ABCDEF'

; Строковые константы для форматирования вывода

SPACE DB ' ' ; Пробел для разделения символов

EQUAL_SIGN DB '=' ; Строка для вывода перед шестнадцатеричным представлением

; Флаг для управления выходом из программы (1 = завершить программу)

EXIT_FLAG DB 0

DTSEG ENDS

MYCODE SEGMENT 'CODE'

ASSUME CS:MYCODE, DS:DTSEG, SS:STSEG ; Настройка сегментных регистров

START:

; Инициализация сегмента данных

MOV AX, DTSEG

MOV DS, AX

; Инициализация счётчика строк

MOV LP, 0

; Очистка экрана перед началом работы

CALL CLRSCR

MAIN_LOOP:

; Проверка, достигнут ли лимит строк (10 строк)

CMP LP, 10

JE EXIT_PROGRAM ; Если обработано 10 строк, завершаем программу

; Ввод строки от пользователя

CALL INPUT_STRING

; Проверка флага выхода (если введён символ '*')

CMP EXIT_FLAG, 1

JE EXIT_PROGRAM

; Вывод строки в шестнадцатеричном формате

CALL PRINT

; Увеличение счётчика строк и переход к следующей итерации

INC LP

JMP MAIN_LOOP

EXIT_PROGRAM:

; Перевод строки для красивого вывода

CALL CLRF

; Вывод сообщения о паузе перед завершением

MOV AH, 09h

LEA DX, MSGPAUSE

INT 21h

; Ожидание нажатия любой клавиши

CALL WAIT_KEY

; Завершение программы через DOS-функцию 4Ch

MOV AH, 4Ch

INT 21h

;-----

; Процедура ввода строки с клавиатуры

; Результат: строка сохраняется в BUF, длина — в LEN

;-----

INPUT_STRING PROC

; Сброс флага выхода

MOV EXIT_FLAG, 0

; Вывод приглашения для ввода строки

MOV AH, 09h

LEA DX, MSG

INT 21h

; Настройка ES для работы со строковыми операциями (STOSB)

PUSH DS

POP ES

; Указатель на буфер (DI = адрес BUF)

LEA DI, BUF

MOV CX, 0 ; Счётчик введённых символов

INPUT_LOOP:

; Чтение символа с клавиатуры

CALL GETCH

; Проверка на символ завершения ввода ('\$')

CMP AL, '\$'

JE END_INPUT

; Обработка Backspace (удаление последнего символа)

CMP AL, 8

JE BKSP

; Проверка первого символа на '*' (флаг выхода)

CMP CX, 0

JNE STORE_CHAR

CMP AL, '*'

JNE STORE_CHAR

```
MOV EXIT_FLAG, 1
```

```
JMP END_INPUT
```

```
STORE_CHAR:
```

```
; Сохранение символа в буфер и увеличение счётчика
```

```
STOSB
```

```
INC CX
```

```
JMP INPUT_LOOP
```

```
BKSP:
```

```
; Если символов нет, игнорируем Backspace
```

```
CMP CX, 0
```

```
JE INPUT_LOOP
```

```
; Удаление последнего символа из буфера
```

```
DEC DI
```

```
DEC CX
```

```
; Стирание символа на экране (пробел + возврат курсора)
```

```
MOV DL, ' '
```

```
CALL PUTCH
```

```
MOV DL, 8
```

```
CALL PUTCH
```

```
JMP INPUT_LOOP
```

```
END_INPUT:
```

```
; Сохранение длины строки
```

```
MOV LEN, CX
```

```
RET
```

```
INPUT_STRING ENDP
```

```
;-----
```

```
; Процедура вывода строки в шестнадцатеричном формате
```

; Формат: " = XX XX XX ..."

;-----

PRINT PROC

; Вывод разделителя " = "

MOV AH, 09h

LEA DX, EQUAL_SIGN

INT 21h

; Настройка цикла вывода (CX = длина строки, SI = указатель на BUF)

MOV CX, LEN

LEA SI, BUF

PRINT_LOOP:

; Загрузка символа из буфера

LODSB

; Вывод символа в HEX-формате

MOV DL, AL

CALL HEX

; Вывод пробела между HEX-значениями

MOV AH, 09h

LEA DX, SPACE

INT 21h

LOOP PRINT_LOOP

; Перевод строки после вывода всех символов

CALL CLRF

RET

PRINT ENDP

;-----

; Процедура перевода байта в шестнадцатеричный формат и вывода

; Вход: DL = байт для преобразования

;-----

HEX PROC

PUSH AX

PUSH DX

; Преобразование старшего полубайта

MOV AL, DL

SHR AL, 4 ; Сдвиг вправо на 4 бита

LEA BX, TABLHEX ; Загрузка таблицы символов HEX

XLAT ; Преобразование AL в символ HEX

MOV DL, AL

CALL PUTCH ; Вывод символа

; Преобразование младшего полубайта

POP DX ; Восстановление исходного байта

MOV AL, DL

AND AL, 0Fh ; Маскирование старшего полубайта

XLAT

MOV DL, AL

CALL PUTCH

POP AX

RET

HEX ENDP

;-----

; Процедура вывода символа (из DL) через DOS-функцию

;-----

PUTCH PROC

PUSH AX

MOV AH, 02h

INT 21h

POP AX

```

    RET
PUTCH ENDP

```

```

;-----
; Процедура перевода строки (CR + LF)
;-----

```

```

CLRF PROC
    PUSH DX
    MOV DL, 13 ; Возврат каретки
    CALL PUTCH
    MOV DL, 10 ; Перевод строки
    CALL PUTCH
    POP DX
    RET
CLRF ENDP

```

```

;-----
; Процедура ожидания нажатия любой клавиши
;-----

```

```

WAIT_KEY PROC
    MOV AH, 01h
    INT 21h
    RET
WAIT_KEY ENDP

```

```

;-----
; Процедура чтения символа с клавиатуры (без эха)
; Результат: AL = введённый символ
;-----

```

```

GETCH PROC
    MOV AH, 01h
    INT 21h
    RET
GETCH ENDP

```

;-----

; Процедура очистки экрана через BIOS

;-----

CLRSCR PROC

MOV AH, 06h ; Функция прокрутки экрана вверх

MOV AL, 00h ; Очистка всего экрана

MOV BH, 07h ; Атрибут (белый текст на чёрном фоне)

MOV CX, 0000h ; Верхний левый угол (0,0)

MOV DX, 184Fh ; Нижний правый угол (24,79)

INT 10h

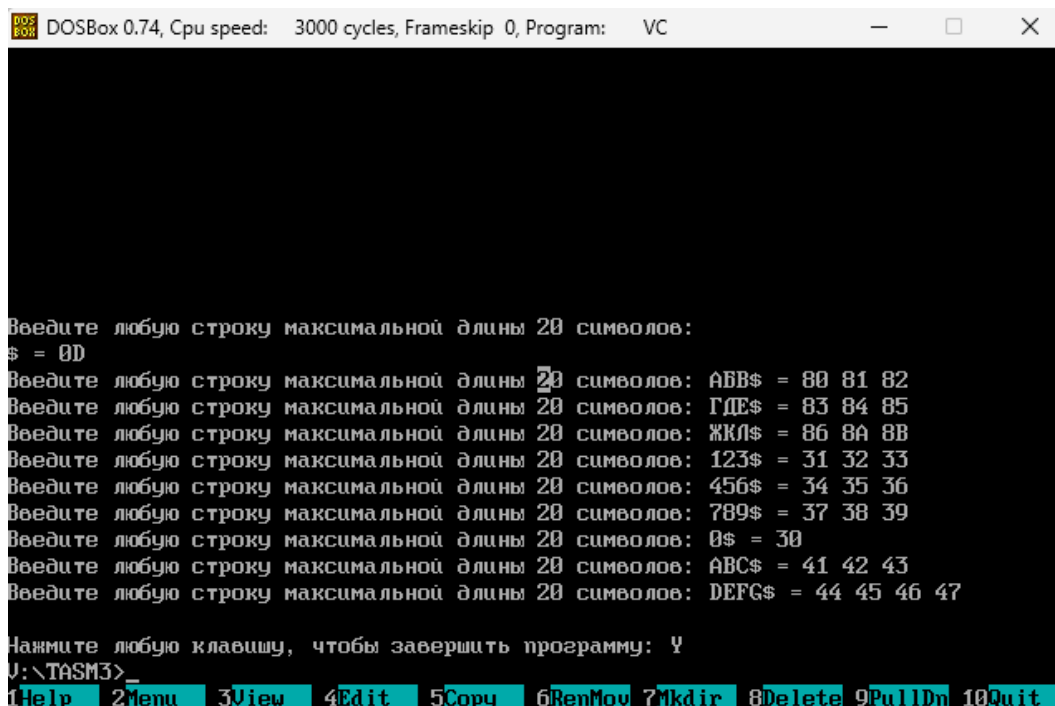
RET

CLRSCR ENDP

MYCODE ENDS

END START

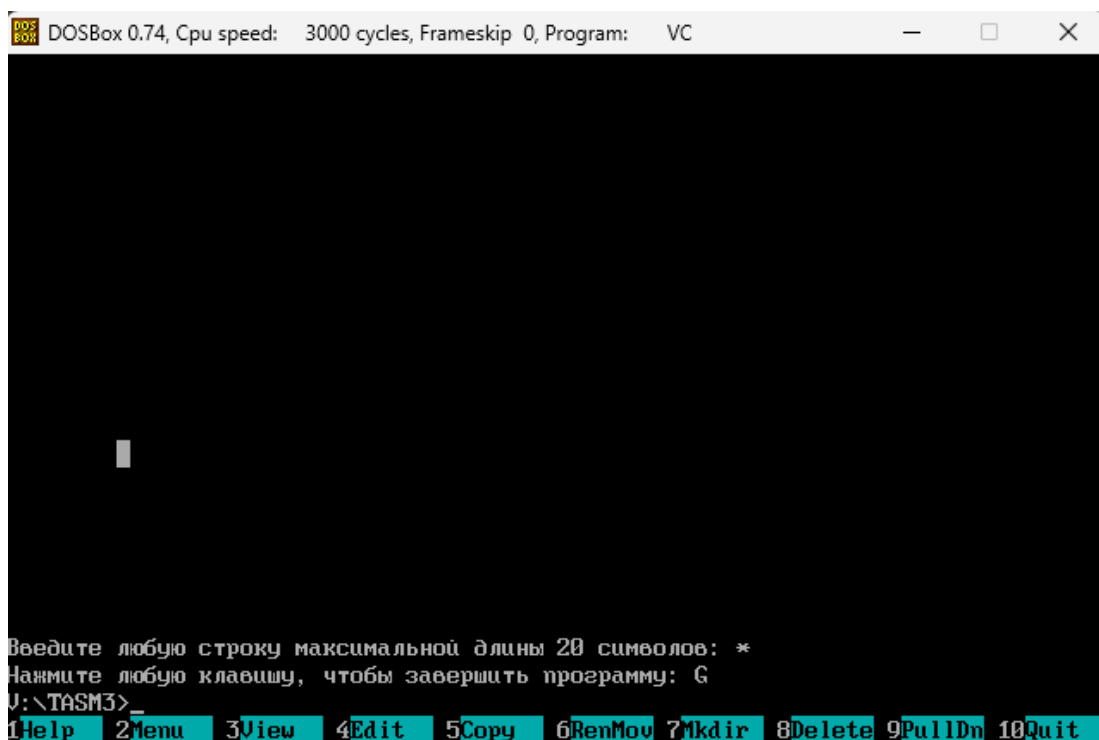
7. Результаты работы программы



```

DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: VC
Введите любую строку максимальной длины 20 символов:
$ = 0D
Введите любую строку максимальной длины 20 символов: АБВ$ = 80 81 82
Введите любую строку максимальной длины 20 символов: ГДЕ$ = 83 84 85
Введите любую строку максимальной длины 20 символов: ЖКЛ$ = 86 8A 8B
Введите любую строку максимальной длины 20 символов: 123$ = 31 32 33
Введите любую строку максимальной длины 20 символов: 456$ = 34 35 36
Введите любую строку максимальной длины 20 символов: 789$ = 37 38 39
Введите любую строку максимальной длины 20 символов: 0$ = 30
Введите любую строку максимальной длины 20 символов: ABC$ = 41 42 43
Введите любую строку максимальной длины 20 символов: DEFG$ = 44 45 46 47
Нажмите любую клавишу, чтобы завершить программу: Y
J:\TASM3>
1Help 2Menu 3View 4Edit 5Copy 6RenMov 7Mkdir 8Delete 9PullDn 10Quit

```



Turbo Assembler Version 3.1 04/13/25 15:46:41 Page 1

lab5_1.asm

```

1  0000                      STSEG SEGMENT STACK 'STACK'
2  0000 0100*(????)          DW 100h DUP(?)      ; Определение сегмента стека
размером 256 слов (512 байт),      +
3                              инициализированных      неопределенными      значениями
4  0200                      STSEG ENDS
5
6  0000                      DTSEG SEGMENT 'DATA'
7                              ;Лабораторная работа №5, Кудабеева Э.,      Группа ИУ5-
42Б
8                              ; Сообщения для пользователя
9  0000 82 A2 A5 A4 A8 E2      A5+      MSG DB      'Введите любую строку максимальной
длины 20 символов: $' ; Приглашение для ввода      +
10      20 AB EE A1 E3 EE      20+ строки
11      E1 E2 E0 AE AA E3      20+
12      AC A0 AA E1 A8 AC      A0+
13      AB EC AD AE A9 20      A4+

```

```

14    AB A8 AD EB 20 32    30+
15    20 E1 A8 AC A2 AE    AB+
16    AE A2 3A 20 24
17    0036 8D A0 A6 AC A8 E2    A5+    MSGPAUSE DB 'Нажмите любую клавишу, чтобы
завершить программу:$' ; Сообщение перед    +
18    20 AB EE A1 E3 EE    20+    завершением программы
19    AA AB A0 A2 A8 E8    E3+
20    2C 20 E7 E2 AE A1    EB+
21    20 A7 A0 A2 A5 E0    E8+
22    A8 E2 EC 20 AF E0    AE+
23    A3 E0 A0 AC AC E3    3A+
24    20 24
25
26                                ; Буфер для хранения вводимой строки (20 символов,
заполнен символами '_' по умолчанию)
27    0069 14*(5F)          BUF DB    20 DUP('_')
28
29                                ; Переменные для хранения данных
30    007D 0000             LEN DW    0 ; Длина введенной строки (в словах)
31    007F 00               LP DB 0 ; Счетчик строк (количество обработанных строк,
максимум 10)
32
33                                ; Таблица для перевода    чисел вшестнадцатеричный
формат
34    0080 30 31 32 33 34 35    36+    TABLHEX DB '0123456789ABCDEF'
35    37 38 39 41 42 43    44+
36    45 46
37
38                                ; Строковые константы для форматирования вывода
39    0090 20 24             SPACE DB ' ' ; Пробел    для разделения символов
40    0092 20 3D 20 24       EQUAL_SIGN DB '=' ;    Строка для вывода перед
шестнадцатеричным представлением
41

```



```

42                                     ; Флаг для управления выходом из программы (1 =
завершить программу)
43 0096 00                               EXIT_FLAG DB 0
44 0097                               DTSEG ENDS
45
46 0000                               MYCODE SEGMENT 'CODE'
47                                     ASSUME    CS:MYCODE, DS:DTSEG, SS:STSEG ;
        Настройка сегментных регистров
48
49 0000                               START:
50                                     ; Инициализация сегмента данных
51 0000 B8 0000s                       MOV AX, DTSEG
52 0003 8E D8                           MOV DS, AX
53
54                                     ; Инициализация счетчика строк
55 0005 C6 06 007Fr 00                 MOV LP, 0
56
57                                     ; Очистка экрана перед      началом      работы
Turbo Assembler      Version 3.1      04/13/25 15:46:41      Page 2
lab5_1.asm

58 000A E8 00D8                       CALL CLRSCR
59
60 000D                               MAIN_LOOP:
61                                     ; Проверка, достигнут ли лимит      строк (10 строк)
62 000D 80 3E 007Fr 0A                 CMP LP, 10
63 0012 74 13                           JE EXIT_PROGRAM ; Если      обработано 10 строк,
завершаем программу
64
65                                     ; Ввод строки от пользователя
66 0014 E8 0021                       CALL INPUT_STRING
67
68                                     ; Проверка флага выхода (если введен символ '*')
69 0017 80 3E 0096r 01                 CMP EXIT_FLAG, 1

```

```

70 001C 74 09          JE EXIT_PROGRAM
71
72                    ; Вывод строки          в шестнадцатеричном формате
73 001E E8 0063        CALL PRINT
74
75                    ; Увеличение счетчика строк и переход к следующей
итерации
76 0021 FE 06 007Fr    INC LP
77 0025 EB E6          JMP MAIN_LOOP
78
79 0027                EXIT_PROGRAM:
80                    ; Перевод строки для красивого      вывода
81 0027 E8 00A4        CALL CLRF
82
83                    ; Вывод сообщения о паузе перед завершением
84 002A B4 09          MOV AH, 09h
85 002C BA 0036r       LEA DX, MSGPAUSE
86 002F CD 21          INT 21h
87
88                    ; Ожидание нажатия любой клавиши
89 0031 E8 00A7        CALL WAIT_KEY
90
91                    ; Завершение программы      через DOS-функцию 4Ch
92 0034 B4 4C          MOV AH, 4Ch
93 0036 CD 21          INT 21h
94
95                    ;-----
96                    ; Процедура ввода строки с      клавиатуры
97                    ; Результат: строка сохраняется в BUF, длина ? в LEN
98                    ;-----
99 0038                INPUT_STRING PROC
100                    ; Сброс флага выхода
101 0038 C6 06 0096r 00 MOV EXIT_FLAG, 0
102

```

```

103                                ; Вывод приглашения для ввода строки
104 003D B4 09                    MOV AH, 09h
105 003F BA 0000r                  LEA DX, MSG
106 0042 CD 21                    INT 21h
107
108                                ; Настройка ES      для работы со строковыми операциями
(STOSB)
109 0044 1E                      PUSH DS
110 0045 07                      POP ES
111
112                                ; Указатель на буфер (DI = адрес BUF)
113 0046 BF 0069r                 LEA DI, BUF
114 0049 B9 0000                 MOV CX, 0 ; Счетчик введенных символов
Turbo Assembler      Version 3.1      04/13/25 15:46:41      Page 3
lab5_1.asm

115
116 004C                        INPUT_LOOP:
117                                ; Чтение символа с клавиатуры
118 004C E8 0091                 CALL GETCH
119
120                                ; Проверка на символ завершения ввода ('$')
121 004F 3C 24                   CMP AL, '$'
122 0051 74 2C                   JE END_INPUT
123
124                                ; Обработка Backspace (удаление последнего символа)
125 0053 3C 08                   CMP AL, 8
126 0055 74 15                   JE BKSP
127
128                                ; Проверка первого символа на '*' (флаг выхода)
129 0057 83 F9 00                 CMP CX, 0
130 005A 75 0C                   JNE STORE_CHAR
131 005C 3C 2A                   CMP AL, '*'
132 005E 75 08                   JNE STORE_CHAR

```

```

133 0060 C6 06 0096r 01      MOV EXIT_FLAG, 1
134 0065 EB 18 90           JMP END_INPUT
135
136 0068                   STORE_CHAR:
137                           ; Сохранение символа в      буфер и      увеличение
счетчика
138 0068 AA               STOSB
139 0069 41               INC CX
140 006A EB E0           JMP INPUT_LOOP
141
142 006C                   BKSP:
143                           ; Если символов нет, игнорируем Backspace
144 006C 83 F9 00         CMP CX, 0
145 006F 74 DB           JE INPUT_LOOP
146
147                           ; Удаление последнего символа из буфера
148 0071 4F               DEC DI
149 0072 49               DEC CX
150
151                           ; Стирание символа на экране (пробел +      возврат
курсора)
152 0073 B2 20           MOV DL, ' '
153 0075 E8 004F         CALL PUTCH
154 0078 B2 08           MOV DL, 8
155 007A E8 004A         CALL PUTCH
156
157 007D EB CD           JMP INPUT_LOOP
158
159 007F                   END_INPUT:
160                           ; Сохранение длины строки
161 007F 89 0E 007Dr      MOV LEN, CX
162 0083 C3               RET
163 0084                   INPUT_STRING ENDP
164

```

```

165                                     ;-----
166                                     ; Процедура вывода      строки в шестнадцатеричном формате
167                                     ; Формат: " = XX XX XX ..."
168                                     ;-----
169 0084                                PRINT PROC
170                                     ; Вывод разделителя " = "
171 0084 B4 09                          MOV AH, 09h
Turbo Assembler      Version 3.1      04/13/25 15:46:41      Page 4
lab5_1.asm

172 0086 BA 0092r                      LEA DX, EQUAL_SIGN
173 0089 CD 21                          INT 21h
174
175                                     ; Настройка цикла вывода (CX =      длина строки, SI =
указатель на  BUF)
176 008B 8B 0E 007Dr                   MOV CX, LEN
177 008F BE 0069r                      LEA SI, BUF
178
179 0092                                PRINT_LOOP:
180                                     ; Загрузка символа из буфера
181 0092 AC                              LODSB
182
183                                     ; Вывод символа в HEX-формате
184 0093 8A D0                          MOV DL, AL
185 0095 E8 000D                        CALL HEX
186
187                                     ; Вывод пробела между HEX-значениями
188 0098 B4 09                          MOV AH, 09h
189 009A BA 0090r                      LEA DX, SPACE
190 009D CD 21                          INT 21h
191
192 009F E2 F1                          LOOP PRINT_LOOP
193
194                                     ; Перевод строки после      вывода всех символов

```

```

195 00A1 E8 002A      CALL CLRF
196 00A4 C3          RET
197 00A5             PRINT ENDP
198
199                  ;-----
200                  ; Процедура перевода байта      в шестнадцатеричный формат
и вывода
201                  ; Вход: DL = байт для преобразования
202                  ;-----
203 00A5             HEX PROC
204 00A5 50          PUSH AX
205 00A6 52          PUSH DX
206
207                  ; Преобразование старшего полубайта
208 00A7 8A C2       MOV AL, DL
209 00A9 D0 E8 D0 E8 D0 E8  D0+   SHR AL, 4 ; Сдвиг вправо на 4 бита
210      E8
211 00B1 BB 0080r      LEA BX, TABLHEX ; Загрузка таблицы символов
HEX
212 00B4 D7          XLAT ;      Преобразование AL в символ HEX
213 00B5 8A D0       MOV DL, AL
214 00B7 E8 000D     CALL PUTCH ; Вывод символа
215
216                  ; Преобразование младшего полубайта
217 00BA 5A          POP DX      ; Восстановление исходного байта
218 00BB 8A C2       MOV AL, DL
219 00BD 24 0F       AND AL, 0Fh ; Маскирование старшего полубайта
220 00BF D7          XLAT
221 00C0 8A D0       MOV DL, AL
222 00C2 E8 0002     CALL PUTCH
223
224 00C5 58          POP AX
225 00C6 C3          RET
226 00C7             HEX ENDP

```

```

227
228                                     ;-----
Turbo Assembler      Version 3.1      04/13/25 15:46:41      Page 5
lab5_1.asm

229                                     ; Процедура вывода      символа      (из DL)через DOS-
функцию
230                                     ;-----
231 00C7      PUTCH PROC
232 00C7 50      PUSH AX
233 00C8 B4 02      MOV AH, 02h
234 00CA CD 21      INT 21h
235 00CC 58      POP AX
236 00CD C3      RET
237 00CE      PUTCH ENDP
238
239                                     ;-----
240                                     ; Процедура перевода строки (CR + LF)
241                                     ;-----
242 00CE      CLRF PROC
243 00CE 52      PUSH DX
244 00CF B2 0D      MOV DL, 13 ; Возврат каретки
245 00D1 E8 FFF3      CALL PUTCH
246 00D4 B2 0A      MOV DL, 10 ; Перевод строки
247 00D6 E8 FFEE      CALL PUTCH
248 00D9 5A      POP DX
249 00DA C3      RET
250 00DB      CLRF ENDP
251
252                                     ;-----
253                                     ; Процедура ожидания нажатия любой      клавиши
254                                     ;-----
255 00DB      WAIT_KEY PROC
256 00DB B4 01      MOV AH, 01h

```

```

257 00DD CD 21          INT 21h
258 00DF C3            RET
259 00E0              WAIT_KEY ENDP
260
261                  ;-----
262                  ; Процедура чтения символа      с клавиатуры (без эха)
263                  ; Результат: AL = введенный символ
264                  ;-----
265 00E0              GETCH PROC
266 00E0 B4 01          MOV AH, 01h
267 00E2 CD 21          INT 21h
268 00E4 C3            RET
269 00E5              GETCH ENDP
270
271                  ;-----
272                  ; Процедура очистки экрана      через BIOS
273                  ;-----
274 00E5              CLRSCR PROC
275 00E5 B4 06          MOV AH, 06h ; Функция прокрутки экрана  вверх
276 00E7 B0 00          MOV AL, 00h ; Очистка всего экрана
277 00E9 B7 07          MOV BH, 07h ; Атрибут (белый текст на черном фоне)
278 00EB B9 0000        MOV CX, 0000h ; Верхний левый угол (0,0)
279 00EE BA 184F        MOV DX, 184Fh ; Нижний    правый угол (24,79)
280 00F1 CD 10          INT 10h
281 00F3 C3            RET
282 00F4              CLRSCR ENDP
283
284 00F4              MYCODE ENDS
285                  END START

```


Turbo Assembler Version 3.1 04/13/25 15:46:41 Page 6

Symbol Table

Symbol Name	Type	Value
??DATE	Text	"04/13/25"
??FILENAME	Text	"lab5_1 "
??TIME	Text	"15:46:41"
??VERSION	Number	030A
@CPU	Text	0101H
@CURSEG	Text	MYCODE
@FILENAME	Text	LAB5_1
@WORDSIZE	Text	2
BKSP	Near	MYCODE:006C
BUF	Byte	DTSEG:0069
CLRF	Near	MYCODE:00CE
CLRSCR	Near	MYCODE:00E5
END_INPUT	Near	MYCODE:007F
EQUAL_SIGN	Byte	DTSEG:0092
EXIT_FLAG	Byte	DTSEG:0096
EXIT_PROGRAM	Near	MYCODE:0027
GETCH	Near	MYCODE:00E0
HEX	Near	MYCODE:00A5
INPUT_LOOP	Near	MYCODE:004C
INPUT_STRING	Near	MYCODE:0038
LEN	Word	DTSEG:007D
LP	Byte	DTSEG:007F
MAIN_LOOP	Near	MYCODE:000D
MSG	Byte	DTSEG:0000
MSGPAUSE	Byte	DTSEG:0036
PRINT	Near	MYCODE:0084
PRINT_LOOP	Near	MYCODE:0092
PUTCH	Near	MYCODE:00C7
SPACE	Byte	DTSEG:0090
START	Near	MYCODE:0000
STORE_CHAR	Near	MYCODE:0068

TABLHEX	Byte	DTSEG:0080
WAIT_KEY	Near	MYCODE:00DB

Groups & Segments	Bit	Size	Align	Combine	Class
-------------------	-----	------	-------	---------	-------

DTSEG	16	0097	Para	none	DATA
MYCODE	16	00F4	Para	none	CODE
STSEG	16	0200	Para	Stack	STACK

8. Выводы по ЛР № 5

В ходе выполнения лабораторной работы были получены следующие навыки:

- освоены методы работы с массивами и строками в Ассемблере, включая буферизацию ввода с использованием команд STOSB и LODSB,
- реализована программа, которая корректно сохраняет введенные символы в буфер и преобразует их в шестнадцатеричный вид с помощью процедуры HEX.